

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must be typeset (L^AT_EX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Asymptotic Notation (25 points)

For each of the following statements say if it true or false and prove your answer. The base of log is 2 unless otherwise specified, and $\ln$ is $\log_e$.

- (a) $n \tan n = O(2^n)$
- (b) $e^n = \Theta(2^n)$
- (c) $n \cos n = O(n)$
- (d) $3^n = \Omega(3^{(n+2)})$
- (e) $\log(n^{1/5}) = \Theta(\log(n^3))$
- (f) Let f, g be positive functions. Then $f(n) + g(n) = \Omega(\max(f(n), g(n)))$

2 Recurrences (25 pts)

Solve the following recurrences, giving your answer in Θ notation (so prove both an upper bound and a lower bound). For each of them you may assume $T(x) = 1$ for $x \leq 5$. Justify your answer (formal proof not necessary, but recommended).

- (a) $T(n) = 5T(n - 3)$
- (b) $T(n) = n^{1/4}T(n^{3/4}) + n$
- (c) $T(n) = 6T(n/4) + n$

3 Group Sorting (50 points)

We say that an array A of size n is *k-group sorted* if it can be divided into k consecutive groups, each of size n/k , such that the elements in each group are larger than the elements in earlier groups, and smaller than elements in later groups. The elements within each group need not be sorted.

For example, the following array is 4-group sorted:

1	2	4	3	7	6	8	5	10	11	9	12	15	13	16	14
---	---	---	---	---	---	---	---	----	----	---	----	----	----	----	----

Note that every array is 1-group-sorted, and only sorted arrays are n -group sorted. You may assume that all elements are distinct, and if you want to you may assume that n and k are powers of 2.

Describe an algorithm that k -group-sorts an array in (worst-case) time $O(n \log k)$. In other words, in at most $O(n \log k)$ time it must turn an array which is not k -group sorted into one that is. Prove correctness and running time.